Variables

…it's a container!

- A Variable is container that stores data, so that data can be retrieved for later use. The data can be (not an exhaustive list):
  - XML
  - A Collection
  - An Object
  - A String or Integer/Float
- Variables in PS do not need to be declared (but they can be).
- And they can be reused! The `$var` can hold one thing for a moment and then something completely different another moment.

Objectification!

- ***Everything*** in PS is an object (even strings).
- Sometimes objects are referred to simply as ***values***
  (because the value of an object is usually what we're interested in)
- While we are usually interested in the value, here's an example that shows we can still manipulate a string like an object.
- Consider:
  - `"Pickle" | GM`
  - Outputs a string abject with many properties and methods.
  - `("Pickle").ToUpper()` ← Which we can then manipulate.

"Pickle", variablized: Store the object to be used later…

- `$st = "Pickle"`
- `$st.ToUpper()`

How to Variablize an object

- We can store the objects values in variables.
  - Use `$<variablesName>`
  - Followed by an *assignment operator* (=)
  - followed by whatever the $var is equal to
- Example:

```
PS C:\Windows\System32> $srv = .\HOSTNAME.EXE
PS C:\Windows\System32> $srv
Server-TDE
PS C:\Windows\System32> _
```

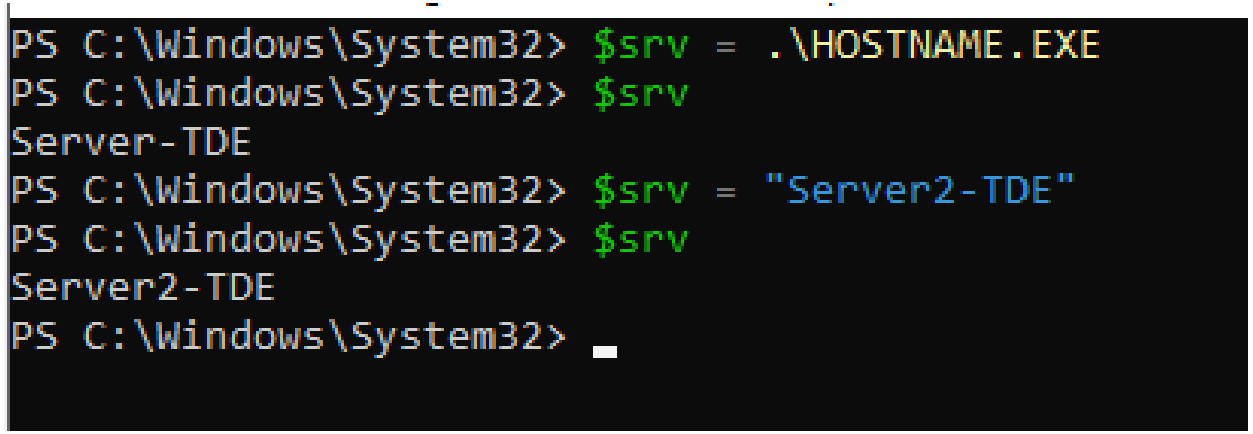
- It is important to note that $ is *not* part of the variable name.
- The $ tells the shell that "what's next is a variable name".

About Variables Names

- $var Names usually contain
  - Letters, and Numbers
  - And its common for them to contain an underscore (_)
- If a $var name contains a space, it must be curly braced { }
  `${Var Name with Spaces}`
- AVOID USING SPACES! It just complicates things.
  (camelCase instead)
  - `$varNameWithSpaces`
- $vars and their values are destroyed when the PS session closes
- Use *meaningful*, as-short-as-possible names for your $vars. Anyone who has to look at your code will thank you.

About Variables Names

- $vars in Powershell can be reused. You can write a value to it and then redeclare the value and it will change.



You can quote me on this…

- Be careful when using quotes with variables.

```
PS C:\Windows\System32> $srv = "Server-TDE"
PS C:\Windows\System32> "Please log into $srv "
Please log into Server-TDE
PS C:\Windows\System32> 'Please log into $srv '
Please log into $srv
PS C:\Windows\System32>
```

- ‘ ‘ makes the string *literal*, ignoring any special characters as special
- “ “ defines a string but *observes* the special characters as special.

Another way to ignore variables in strings…

- ` ← Backtick is the *escape character* of PS. The escape character removes the special meaning from anything that follows. It's also used to **format strings**.

```
PS C:\Windows\System32> "Please log into $srv"
Please log into Server-TDE
PS C:\Windows\System32> "Please log into `$srv"
Please log into $srv
PS C:\Windows\System32>
```

- We can see that, while Using " ", the addition of the ` before `$srv` makes PS ignore the fact that this is a variable.

The Great Escape…character.

- `n (the escape character + the letter n) means "new line" in PS.
- When we put it between two parts of a string, it will separate them into lines.

```
PS C:\workSrv1> $list = "Groceries`nDishes`nLaundry`nDog Walk"
PS C:\workSrv1> $list
Groceries
Dishes
Laundry
Dog Walk
PS C:\workSrv1> _
```

"Groceries`nDishes`nLaundry`nDog Walk"

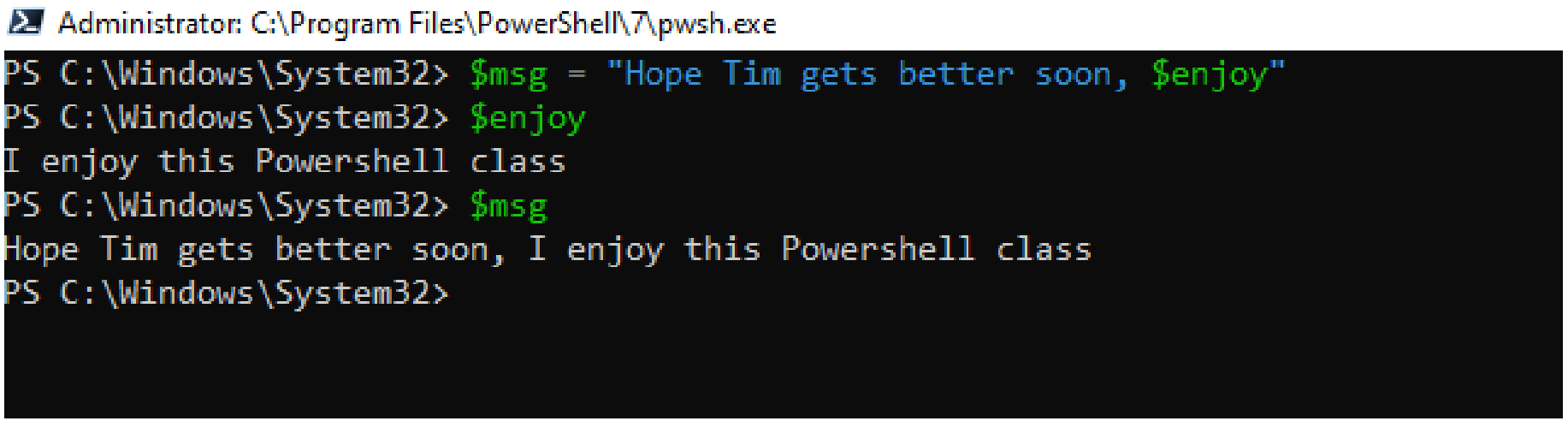
- There are quite a few of these escape characters. Reference `about_special_characters` help files for more details.

Common ways to…escape

Sequence	Description
`0	Null
`a	Alert
`b	Backspace
`e	Escape
`f	Form feed
`n	New line
`r	Carriage return
`t	Horizontal tab
`u{x}	Unicode escape sequence
`v	Vertical tab

Passing the contents of one Var to another

- We can reference $vars inside of other $vars!



Storing more than one object in a $var

- Using a comma separated list (Array) of items.

```
PS C:\Windows\System32> $msg
Hope Tim gets better soon, I enjoy this Powershell class
PS C:\Windows\System32> $computers='Server-TDE','TIM-CLIENT','TIM-CLIENT2'
PS C:\Windows\System32> $computers
Server-TDE
TIM-CLIENT
TIM-CLIENT2
PS C:\Windows\System32>
```

- Note: in this example the commas *cannot* be in the quotes or else our string will be a single item

wrong

```
PS C:\Windows\System32> $computers='Server-TDE,TIM-CLIENT,TIM-CLIENT2'
PS C:\Windows\System32> $computers
Server-TDE,TIM-CLIENT,TIM-CLIENT2
PS C:\Windows\System32>
```

Accessing Properties/Methods of objects

- We can also manipulate an object stored in a $var by accessing its properties and methods (run a **GM** against a command to see what they are).

```
PS C:\Windows\System32> $computers | gm

   TypeName: System.String

Name                 MemberType
----                 ----------
Clone                Method
CompareTo            Method
Contains             Method
CopyTo               Method
```

```
PS C:\Windows\System32> $computers.ToUpper()
SERVER-TDE,TIM-CLIENT,TIM-CLIENT2
```

```
PS C:\Windows\System32> $computers.ToLower()
server-tde,tim-client,tim-client2
PS C:\Windows\System32>
```

Referencing multiple objects from a $var array

- We can then reference each item inside our $var by using its index number.
  - `[0],[1]…[x]` ←Reference the index numbers forwards.
  - `[-1],[-2]` ← Reference the index numbers backwards.

```
PS C:\Windows\System32> $computers='Server-TDE','TIM-CLIENT','TIM-CLIENT2'
PS C:\Windows\System32> $computers[0]
Server-TDE
PS C:\Windows\System32> $computers[1]
TIM-CLIENT
PS C:\Windows\System32> $computers[2]
TIM-CLIENT2
PS C:\Windows\System32> $computers[-1]
TIM-CLIENT2
PS C:\Windows\System32>
```

Working with multiple variable objects

- What if we are storing strings AND integers in the same variable? Or, we just want to do something with ONE of the variables and not the other? We can use the index numbers to specify which variable to modify.

```
PS C:\Windows\System32> $computers='Server-TDE','TIM-CLIENT','TIM-CLIENT2'
PS C:\Windows\System32> $computers[2].Replace('CLIENT2','NEWCLIENT')
TIM-NEWCLIENT
PS C:\Windows\System32>
```

- This is also useful if the $var is holding different object types (one method won't work on the other)

```
PS C:\workSrv1> $multiobj = 100,'one hundred'
PS C:\workSrv1> $multiobj[0].GetHashCode()
100
PS C:\workSrv1> $multiobj[1].ToUpper()
ONE HUNDRED
```

Replacing the value of a variable

- Note: the previous examples (modifying the objects in a variable) does not write the new object value back to the variable (it simply creates a new, temporary value and outputs that to the screen (it's not stored anywhere).
- If we want to dynamically *change* the value held in the variable to the new value produced, we do it like this.

```
PS C:\Windows\System32> $computers[2] = $computers[2].Replace('CLIENT2','NEWCLIENT')
PS C:\Windows\System32> $computers
Server-TDE
TIM-CLIENT
TIM-NEWCLIENT
```

Working with multiple $var objects (cont'd)

- What if I have multiple objects in my $var and I want to run a method against each one at the same time (instead of a separate statement for each)?
- For everything in `$computers`, look at what's in the pipeline (`$_.`)
- For each object in the pipeline, apply its `toLower` method.

```
PS C:\Windows\System32> $computers | ForEach-Object {$_.ToLower()}
server-tde
tim-client
tim-newclient
```

Working with multiple $var objects (cont'd)

- If we want to write those values back to the variable

```
PS C:\Windows\System32> $computers = $computers | ForEach-Object {$_.ToLower()}
PS C:\Windows\System32> $computers
server-tde
tim-client
tim-newclient
PS C:\Windows\System32>
```

{ } = scriptblock
$_. = pipeline placeholder

More than one way to skin a command…wut?

- You may have noticed that some of the ways we write commands overlap. This is because you're starting to understand how .net works.
- Consider these two commands. They are *identical* in function.

```
PS C:\workSrv1> Get-Service | ForEach-Object {Write-Host $_.Name}
PS C:\workSrv1> Get-Service | Select-Object -ExpandProperty Name
```

- It really pays to pay attention to *how .net works* and not just writing PS syntax.

Using an object property in a string.

- Consider this example first. We are pulling the length property from our string.

```
PS C:\Windows\System32> $tim = $computers.Length
PS C:\Windows\System32> $compLeng = $computers.Length
PS C:\Windows\System32> $compLeng
3
```

- To use this info in-line in a string of text we must use a *subexpression* $().

```
PS C:\Windows\System32> $tim = "The number of computers is $($computers.Length)"
PS C:\Windows\System32> $tim
The number of computers is 3
```

Declaring a $vars type

- PS works with data well enough to interpret how to use different data types in different situations, but sometimes you must *specify* the type of data in the $var.

```
PS C:\workSrv1> $num = Read-host "Enter a number from 1 - 10"
Enter a number from 1 - 10: 10
PS C:\workSrv1> $num * 10
10101010101010101010
```

- That's not what we meant PS! PS did its best and said: "Oh you want 10 "10's" (the string not the number), here's 10 10's.

:/

Declaring a $vars type

- There's the problem. It's a string. We can fix this by *Declaring the $var type* during its creation. Like this:

```
PS C:\workSrv1> $num | GM

   TypeName: System.String
```

```
PS C:\workSrv1> [int]$num = Read-host "Enter a number from 1 - 10"
Enter a number from 1 - 10: 10
PS C:\workSrv1> $num * 10
100
```

Variable types in PS.

- `[int]` – Integers (non-decimal numbers)
- `[single][double]` – floating numbers (numbers w/e a decimal)
- `[string]` – A string of Characters
- `[char]` – Exactly one Character
- `[xml]` – and XML specification document
- `[adsi]` – AD Service Interface queries (Puts AD objects in the $var)

```
PS C:\workSrv1> [int]$num = Read-host "Enter a number from 1 - 10"
Enter a number from 1 - 10: dog
MetadataError: Cannot convert value "dog" to type "System.Int32". Error:
"Input string was not in a correct format."
```

Variable Management commands.

- `Clear-Variable` — Deletes the value of a variable.
- `Get-Variable` — Gets the variables in the current console.
- `New-Variable` — Creates a new variable.
- `Remove-Variable` — Deletes a variable and its value.
  (probably the only one you'll use)
- `Set-Variable` — Changes the value of a variable.

The Rules

- Variable names should be:
  - Simple
  - Descriptive
  - Single word, camel-cased names (`$dogFood`, not `${Dog Food}`)
- If the variable contains only one type of object, declare the type to prevent errors.
- Get in the habit of declaring variable types.